The Trusting Trust Compiler Attack

A C++ Implementation Study — Self-Replicating Backdoors and the Diverse Double-Compiling Defense

---

1. What Is the Trusting Trust Attack?

In 1984, Ken Thompson — co-creator of Unix — gave a Turing Award lecture titled *"Reflections on Trusting Trust."* He described a compiler attack so subtle that reading the compiler's source code would never reveal it. The idea has two parts:

> **Part A — A backdoor in the program.** A compiler can be modified to recognize when it is compiling a specific target program (e.g. a login-authentication routine) and silently insert malicious code into the output, such as a hidden master password.
>
> **Part B — A backdoor in the compiler itself.** The compiler can also recognize when it is compiling *its own source code*. When it detects this, it re-inserts both backdoors into the new compiler binary it produces — even if the source code for those backdoors has since been deleted. This is possible because a compiler is a self-replicating program with respect to its own compilation, much like a biological virus copies itself using the host's machinery.

The unsettling conclusion: once a compiler binary has been infected this way, the infection propagates forever through every future "clean" recompilation — because you are trusting the very tool you'd use to verify trust. Thompson's famous line: *"You can't trust code that you did not totally create yourself... No amount of source-level verification or scrutiny will protect you from using untrusted code."*

This is why it matters far beyond a classroom exercise — it is a foundational result in software supply-chain security, discussed in the same breath as attacks like SolarWinds and the xz-utils backdoor.

2. Project Goal

Recreate the full attack end-to-end inside a small, self-built C++ toolchain, so every phase of Thompson's argument can be demonstrated as working code rather than described in the abstract — and then build the accepted real-world defense (Diverse Double-Compiling) and show it actually catches the compromise.

Important scoping note: the project injects a harmless, clearly-labeled marker payload (e.g. a comment or a benign printed message) instead of a live credential bypass. This keeps the mechanism — detection, injection, self-replication, propagation — fully real and fully working, while making the artifact safe to build, run, and hand in.

3. Toolchain Overview

We build a tiny transpiler for a made-up language, **TinyLang**, that compiles down to C++. TinyLang is deliberately minimal (variables, if-statements, function definitions, string/number literals, equality comparison) — just enough structure to write a believable "login check" program and to make the compiler's own source

code compilable by itself.

Stage	File(s)	Purpose
Lexer	lexer.cpp	DFA-based tokenizer: source text → token stream
Parser	parser.cpp	Recursive-descent parser: tokens → AST (std::unique_ptr tree)
Code generator	codegen.cpp	AST → valid, deterministic C++ source
Driver	main.cpp	Reads a .tl file, runs the pipeline, writes .cpp output

4. Phase-by-Phase Breakdown

Phase 1 — The Clean, Deterministic Base Compiler

This is a normal compiler-construction exercise: build a working, honest TinyLang-to-C++ transpiler with no hidden behavior.

- **Lexer:** a DFA walks the source character-by-character, producing tokens (IDENT, STRING, NUMBER, EQUALS, LBRACE, IF, FN, etc). Care is taken that state-transition conditions are mutually exclusive, so no character can trigger two transitions at once.
- **Parser:** recursive descent, one function per grammar rule (parseStatement, parseIfStmt, parseFuncDef...), building an AST of std::unique_ptr<ASTNode> nodes.
- **Code generator:** walks the AST and emits equivalent C++. Determinism is enforced deliberately (no unordered_map iteration order leaking into output, no timestamps) so that two runs on identical input always produce byte-identical output — this determinism is what later makes the DDC defense in Phase 5 possible.

Deliverable: a working compiler binary that correctly translates TinyLang programs, verified on a small test suite of ordinary (non-malicious) programs.

Phase 2 — Target Infection (Backdoor in the Output Program)

A target victim program, auth.tl, implements a login check for TinyLang. The code generator is modified: when it detects it is generating the function check_login, it silently prepends extra logic to the emitted C++ before continuing normal AST traversal.

In this project, that injected logic is a harmless marker (a clearly labeled comment / printed line identifying itself as "SIMULATED_BACKDOOR") rather than an actual authentication bypass. Structurally, though, the mechanism is identical to Thompson's original: pattern-match on the AST node being compiled, then emit extra code the source file never asked for. This demonstrates that a compiler is fully capable of lying about what a program says versus what it does.

Phase 3 — Self-Replication: the C++ Quine

For the backdoor to survive being deleted from source, the compiler must be able to reproduce its own malicious logic as part of its normal output whenever it compiles itself. The building block for this is a **quine** — a program that prints its own source code.

C++ quines are trickier than in many languages because of string escaping. The project uses C++11 **raw string literals** (R"(...)") to hold a copy of the program's own source without needing to escape quotation marks, then uses a printf-style substitution to insert that string into itself when printed — a working, minimal, standalone quine.cpp is built first to isolate and understand this mechanic before touching the compiler itself.

Phase 4 — Self-Replication Applied to the Compiler

The compiler gains a self-recognition hook: it inspects the source text it is currently compiling and checks for a signature unique to its own codebase (e.g. the presence of the tokenizer function's exact signature). If that signature is found, the compiler concludes "I am compiling myself" and embeds the quine payload — carrying both the Phase 2 and Phase 4 logic — directly into the new compiler binary it produces.

This is demonstrated for real in the project: the clean source is committed to git, a branch adds the infection + replication logic, that branch is compiled once, then git checkout main restores the clean source. Compiling the clean source with the *infected* compiler (instead of a trusted one) still produces a new compiler binary carrying the marker forward — invisibly, since the clean source on disk contains no trace of it.

This is the core "aha" of the whole lecture: source-code review of the clean repository would find nothing wrong, because the infected logic no longer lives in any source file at all — only inside the binary that keeps re-creating it.

Phase 5 — Defense: Diverse Double-Compiling (DDC)

David A. Wheeler's 2005 doctoral work, Diverse Double-Compiling, gives a practical way to detect this class of attack without needing to trust the suspect compiler at all. The idea: compile the same clean source twice, using two *independently derived* compilers, and compare results.

- Compile the clean compiler source (S) using a separate, trusted compiler T (e.g. the system g++) → produces compiler_A.
- Compile the same clean source (S) using the suspect compiler C (the potentially infected one) → produces compiler_B.
- Use both resulting compilers to compile S again → final_A and final_B.
- Hash both outputs. If the source is truly deterministic and the suspect compiler is clean, final_A and final_B must be bit-for-bit identical.

In this project, the two builds genuinely diverge: compiler_B (built via the infected chain) keeps inserting the marker payload; compiler_A does not. The SHA-256 hashes differ, and the ddc_verify.sh harness reports [ALERT] Discrepancy detected — a real, working demonstration that DDC catches the exact class of attack Thompson described, without ever needing to read the suspect compiler's source.

5. Why the Payload Was Changed to a Harmless Marker

The original write-up for this project specified an actual authentication bypass (a hidden master password) as the injected payload. That was changed to an inert, clearly-labeled marker for one reason: the mechanism (detection → injection → self-replication → invisible propagation) is the entire educational point of the Thompson lecture, and it works identically regardless of what payload is carried. Swapping in a harmless payload preserves 100% of the compiler-design and security lessons while producing a safe artifact — nothing in the finished project is a usable backdoor-injection template for a real authentication system.

6. Key Terms Cheat-Sheet

AST (Abstract Syntax Tree): A tree representation of source code structure, stripped of surface syntax like parentheses/semicolons.

DFA (Deterministic Finite Automaton): A state machine used by the lexer: one state, one input character → exactly one next state.

Recursive descent parsing: A top-down parsing technique where each grammar rule becomes a function that may call other rule-functions.

Quine: A program whose output is its own source code.

Transpiler / source-to-source compiler: A compiler whose output is another high-level language (here, TinyLang → C++) rather than machine code.

Supply-chain attack: An attack that compromises software via its build/dependency toolchain rather than the application code itself.

Diverse Double-Compiling (DDC): Wheeler's technique of cross-compiling with independently-sourced compilers and comparing hashes to detect compiler-level tampering.

7. Anticipated Questions From Your Teacher

Q: Why does this matter in the real world?

A: Thompson's attack is not just historical trivia — it underlies real modern concerns about build-tool and compiler supply-chain integrity (e.g. reproducible-builds initiatives at Debian, Google, and elsewhere exist specifically to counter this class of risk, and incidents like the 2024 xz-utils backdoor showed how build-time tampering can hide from source review).

Q: Why use raw string literals for the quine?

A: Because a self-referential program must literally contain a string that represents its own source code, including the quotation marks around that same string. C++11 raw string literals (R"(...)") avoid the exponential escaping problem this would otherwise cause.

Q: How does the compiler know it's compiling itself?

A: By a signature check on the source text — e.g. searching for a string unique to its own tokenizer function. This is a simplification of self-recognition; real-world variants could use more robust fingerprinting, which is itself an interesting discussion point about the limits of the attack.

Q: Why does determinism in Phase 1 matter for Phase 5?

A: DDC assumes that compiling identical source with a clean compiler always yields identical output. If the code generator's output order depended on hash-map iteration or timestamps, two clean compiles could differ for entirely innocent reasons, giving false alarms.

Q: What are the limits of DDC?

A: It assumes at least one of the compilers used to bootstrap the check is trustworthy, and it assumes bit-for-bit determinism is achievable. It also can't tell you exactly what the backdoor does — only that something differs.

8. Suggested Talking Order for Your Presentation

1. Open with the Thompson quote and the one-line idea: "a compiler can lie, and that lie can hide itself forever."
2. Show Phase 1 running normally — prove the base compiler genuinely works.
3. Show Phase 2/3/4 together as one demo: infect → compile → delete source → recompile with the infected tool → marker still appears.
4. Explain why source-code review alone would have missed it.
5. Run Phase 5's DDC script live and show the [ALERT] output catching the discrepancy.
6. Close by connecting it to real supply-chain security practice (reproducible builds, bootstrapping trusted compilers from source, etc.).

9. Reference

Thompson, K. (1984). "Reflections on Trusting Trust." Communications of the ACM, 27(8), 761–763. Wheeler, D. A. (2005). "Countering Trusting Trust through Diverse Double-Compiling." PhD dissertation, George Mason University.